

RapidR Compiler & Language Manual

Purpose: This is a comprehensive reference for AI assistants (and humans) working on the RapidR transpiler. It documents the Rust workspace, language syntax, compiler internals, runtime architecture, known limitations, and coding patterns.

Instructions for AI: Always read this manual first when working on this repository. Keep this manual updated as features are added or changed.

Last Updated: April 1, 2026

Table of Contents

- [1. Project Overview](#)
- [1.1 RapidR Bootstrap Status](#)
- [2. Architecture & Pipeline](#)
- [3. Language Syntax Reference](#)
 - [3.1 Comments](#)
 - [3.2 Variables & Types](#)
 - [3.3 Constants](#)
 - [3.4 Arrays](#)
 - [3.5 User-Defined Types \(TYPE\)](#)
 - [3.6 Control Flow](#)
 - [3.7 Subroutines & Functions](#)
 - [3.8 CREATE Blocks \(GUI\)](#)
 - [3.9 WITH Blocks](#)
 - [3.10 IMPORT Statement](#)
 - [3.11 DECLARE Statement \(DLL/FFI\)](#)
 - [3.12 BIND Statement](#)
 - [3.13 PRINT Statement](#)
 - [3.14 Operators](#)
 - [3.15 String Suffixes](#)
- [4. Preprocessor Directives](#)
- [5. Compiler Internals](#)
 - [5.1 Lexer \(rapidr-lexer\)](#)

- 5.2 Parser (rapidr-parser)
- 5.3 AST (rapidr-ast)
- 5.4 Code Generator (rapidr-codegen-rust)
- 5.5 Preprocessor (rapidr-preprocessor)
- 5.6 Diagnostics (rapidr-diagnostics)
- 5.7 Symbol Table
- 5.8 Component Registry
- 6. Runtime Library
 - 6.1 Builtins (builtins.rs)
 - 6.2 GUI (gui.rs)
 - 6.3 Database (database.rs)
 - 6.4 Network (network.rs)
 - 6.5 Data Science (datascience.rs)
- 7. Code Generation Patterns
- 8. Known Limitations & Gotchas
- 9. How to Add New Features
- 10. Test Suite

1. Project Overview

RapidR is a BASIC-to-Rust transpiler that reads `.rr` source files (BASIC-like syntax inspired by RapidQ) and produces standalone Rust projects that compile to native executables. The project directory structure:

<code>crates/</code>	– RapidR Rust workspace crates (8 crates)
<code> rapidr-cli/</code>	– CLI entry point (version, lex, parse, codegen, shortcut build)
<code> rapidr-diagnostics/</code>	– TextSpan, SourceLocation, Diagnostic types
<code> rapidr-ast/</code>	– Shared AST data structures
<code> rapidr-lexer/</code>	– Tokenization (keywords, literals, operators, directives)
<code> rapidr-parser/</code>	– Recursive-descent parser → AST
<code> rapidr-preprocessor/</code>	– <code>\$INCLUDE</code> , <code>\$DEFINE</code> , <code>\$IFDEF</code> , <code>\$MACRO</code> , <code>\$THEME</code>
<code> rapidr-codegen-rust/</code>	– AST → Rust source code generator
<code> rapidr-runtime-core/</code>	– Native runtime (GUI, builtins, database, networking, data sci
<code>examples/</code>	– Demo and test <code>.rr</code> programs
<code>utilities/vscodeext/</code>	– VS Code extension

1.1 RapidR Bootstrap Status

The Rust migration has reached **functional transpiler status** with a complete pipeline from `.rr` source to native executables. The Cargo workspace contains 8 crates:

Crate	Lines	Description
<code>rapidr-cli</code>	—	CLI with <code>version</code> , <code>preprocess</code> , <code>lex</code> , <code>parse</code> , <code>codegen</code> commands
<code>rapidr-diagnostics</code>	—	<code>TextSpan</code> , <code>SourceLocation</code> , and <code>Diagnostic</code> types
<code>rapidr-ast</code>	—	Shared AST data structures
<code>rapidr-preprocessor</code>	—	Directives: <code>\$DEFINE</code> , <code>\$UNDEF</code> , <code>\$IFDEF</code> , <code>\$IFNDEF</code> , <code>\$ELSE</code> , <code>\$ENDIF</code> , <code>\$MACRO</code> , <code>\$INCLUDE</code> , <code>\$THEME</code>
<code>rapidr-lexer</code>	—	Lexer covering keywords, literals, directives, operators, suffixes, line continuations
<code>rapidr-parser</code>	—	Recursive-descent parser producing typed AST
<code>rapidr-codegen-rust</code>	~2,100	Rust code generator — walks AST, emits Rust source targeting <code>rapidr-runtime-core</code>
<code>rapidr-runtime-core</code>	~5,700	Native runtime — FLTK GUI (~2,200 lines), builtins, database (MySQL/SQLite), networking, file I/O

Key architecture decisions:

- Generated code uses `thread_local!` storage for module-level variables (`gv()` / `gs()` scalar accessors, `ga_get()` / `ga_set()` array accessors), correctly sharing state across SUBs/FUNCTIONs
- GUI components use FLTK via the `fltk` crate with `fltk-theme` for theming
- Component properties/methods are dispatched through a centralized `rp_comp_get` / `rp_comp_set` / `rp_comp_method` API backed by thread-local `GUI_COMPONENTS` storage
- UDT variables remain as native Rust structs (not stored in the global variable `HashMap`)

Current validation:

- All Rust unit tests pass across all crates
- All 29 example programs generate, compile, and run

- The self-hosted IDE (`examples/ide.rr`) compiles to a native FLTK application with working properties, events, code view, and design surface

```
cargo test                # Run all tests
cargo run -- codegen examples/ide.rr /tmp/ide_rust # Generate IDE
cd /tmp/ide_rust && cargo build && ./target/debug/ide # Build and run
```

CLI Usage

```
# Shortcut syntax – builds and places binary alongside source
rapidr --release examples/hello_world.rr
rapidr --debug examples/hello_world.rr
```

```
# Full subcommand syntax
rapidr codegen <file.rr> <outdir> [--release|--debug]
rapidr lex <file.rr>
rapidr parse <file.rr>
rapidr preprocess <file.rr>
rapidr version
```

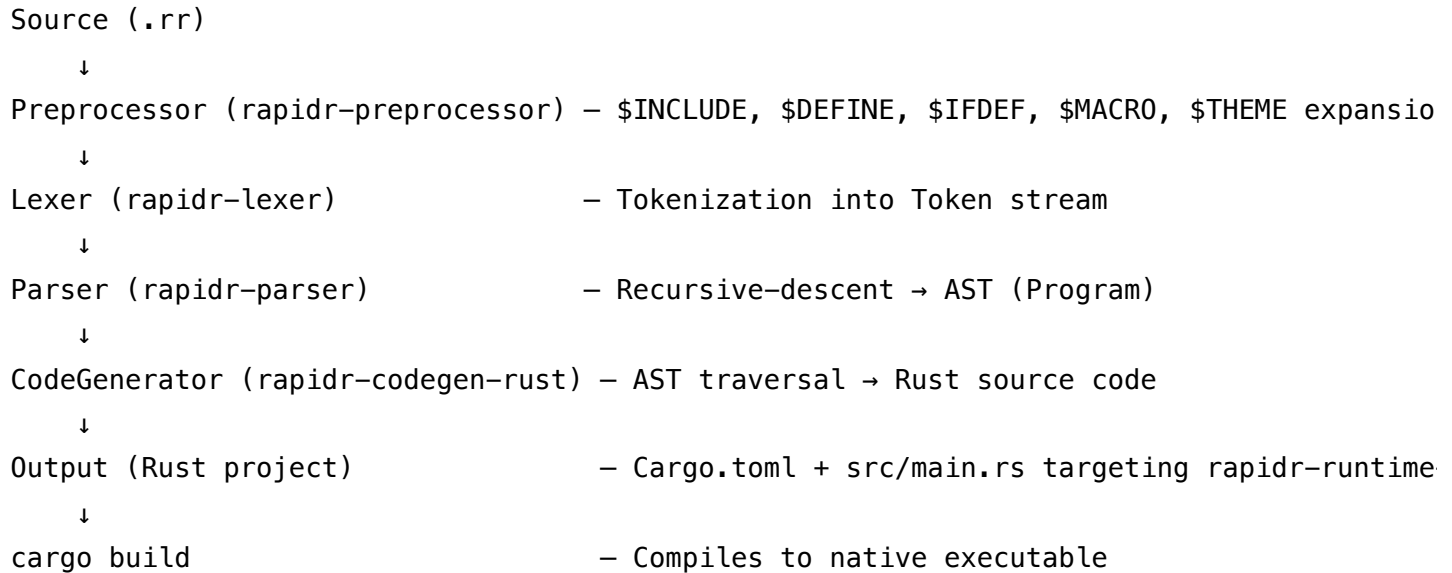
Command	Description
codegen <file.rr> <outdir>	Generate a Rust project from a <code>.rr</code> file
lex <file.rr>	Dump token stream
parse <file.rr>	Dump AST
preprocess <file.rr>	Dump preprocessed source
version	Print version

Optional flags for `codegen` :

Flag	Description
--release	Build in release mode (optimized)
--debug	Build in debug mode (default)

2. Architecture & Pipeline

The compilation pipeline:



Key design decisions:

- The parser is a **single-pass recursive-descent** parser — no separate semantic analysis pass.
- The code generator does **three pre-passes** before generating code:
 - i. Collect UDT type names
 - ii. Collect global variables, arrays, constants, and CREATE components
 - iii. Collect SUB/FUNCTION signatures, DECLARE, IMPORT modules
- Then processes directives (*TYPECHECK*, APPTYPE, \$THEME, etc.)
- Then generates Rust code by visiting AST nodes.

3. Language Syntax Reference

3.1 Comments

```
' This is a comment (single quote)
REM This is also a comment
```

Comments are stripped by the lexer. Everything after ' or REM on a line is ignored.

3.2 Variables & Types

```
DIM x AS INTEGER
DIM name AS STRING
DIM value AS DOUBLE
DIM flag AS LONG
DIM anything AS VARIANT      ' Default type when unspecified
```

Supported types:

Type	Rust Equivalent	Default Value
INTEGER	i64	0
LONG	i64	0
INT64	i64	0
BYTE	i64	0
WORD	i64	0
DWORD	i64	0
SINGLE	f64	0.0
DOUBLE	f64	0.0
CURRENCY	f64	0.0
STRING	String	""
VARIANT	Value	v_null()
ROBJECT	Value	v_null()
Any P-component	component instance	rp_comp_create()
Any UDT name	struct instance	constructor call

Multiple declarations:

```
DIM a, b, c AS INTEGER      ' All three are INTEGER
```

Implicit declaration: When `$TYPECHECK OFF` (default), assigning to an undeclared variable auto-declares it as `DOUBLE` (or whatever `$OPTION DIM` specifies).

Case insensitivity: Identifiers are case-insensitive. The codegen lowercases all identifiers (e.g., `MyVar` $\rightarrow$ `myvar`).

Suffix conventions (legacy BASIC): The lexer accepts `$`, `%`, `#`, `&`, `!` suffixes on identifiers (e.g., `name$`, `count%`). These are stripped during code generation.

3.3 Constants

```
CONST PI = 3.14159
CONST MAX_SIZE AS LONG = 100      ' Optional type annotation (ignored)
CONST GREETING = "Hello"
```

Constants are compiled as regular variable assignments.

3.4 Arrays

```
DIM values(100) AS DOUBLE          ' Array of 101 elements (0-100)
DIM grid(10, 20) AS INTEGER        ' 2D array (currently stored as nested list)
DIM names(1 TO 50) AS STRING       ' 1-based range (allocates 51 elements, 0 unused)
```

Important: In RapidR, `DIM A(10)` creates 11 elements (indices 0-10), matching classic BASIC semantics.

Array access uses parentheses (not square brackets):

```
values(5) = 42.0
x = values(i)
```

The array/function ambiguity: Both array access and function calls use `()` — e.g., `foo(5)` could be either. The parser always creates `FunctionCallNode` at parse time. The codegen resolves this:

1. If the name is in `self.arrays` set $\rightarrow$ emit `name[idx]`
2. If the name is in the symbol table as `variable / array / component` $\rightarrow$ emit `name[idx]`
3. Otherwise $\rightarrow$ emit `name(idx)` (function call)

3.5 User-Defined Types (TYPE)

```
TYPE PersonType
    Name AS STRING
    Age AS INTEGER
    Scores(10) AS DOUBLE
END TYPE

TYPE Employee EXTENDS PersonType
    Department AS STRING

    CONSTRUCTOR
        .Department = "Unknown"
    END CONSTRUCTOR

    SUB Display()
        PRINT .Name + " - " + .Department
    END SUB
END TYPE

DIM p AS PersonType
p.Name = "Alice"
p.Age = 30
p.Scores(1) = 95.5
```

Generated as Rust structs with Default trait implementation.

Supports:

- Fields with optional array dimensions
- EXTENDS for inheritance
- CONSTRUCTOR block (mapped to `__post_init__`)
- Embedded SUB / FUNCTION methods (gets `self` parameter injected)
- PRIVATE / PUBLIC visibility markers (parsed but ignored in codegen)
- PROPERTY SET attribute (parsed but not fully implemented)

3.6 Control Flow

IF...THEN...END IF (Block)

```
IF x > 10 THEN
    PRINT "Large"
ELSEIF x > 5 THEN
    PRINT "Medium"
ELSE
    PRINT "Small"
END IF
```

IF...THEN (Single-line)

```
IF x > 10 THEN PRINT "Large"
IF x > 10 THEN PRINT "A" : PRINT "B" ELSE PRINT "C"
```

Single-line IF handles : as additional THEN-branch statements until ELSE or newline.

FOR...NEXT

```
FOR i = 1 TO 10
    PRINT STR$(i)
NEXT i          ' Variable name after NEXT is optional

FOR j = 10 TO 0 STEP -1
    PRINT STR$(j)
NEXT j
```

Generated as `while` loop in Rust:

```
gs("i", v_int(1));
while (gv("i").rp_le(&v_int(10))).to_bool() {
    rp_print(&[rp_str_func(&gv("i"))], true);
    gs("i", &gv("i") + &v_int(1));
}
```

This correctly handles both positive and negative STEP values, including floating-point steps.

WHILE...WEND

```
WHILE x < 100
    x = x * 2
WEND
```

DO...LOOP

' Pre-condition forms:

```
DO WHILE x < 100
    x = x + 1
LOOP
```

```
DO UNTIL x >= 50
    x = x + 1
LOOP
```

' Post-condition forms:

```
DO
    x = x + 1
LOOP WHILE x < 100
```

```
DO
    x = x + 1
LOOP UNTIL x >= 50
```

SELECT CASE

```
SELECT CASE grade
    CASE "A"
        PRINT "Excellent"
    CASE "B", "C"
        PRINT "Good"
    CASE ELSE
        PRINT "Try harder"
END SELECT
```

Generated as if/elif/else chain with a temporary variable `_select_val_{line}` .

EXIT Statement

```
EXIT FOR      ' → break
EXIT WHILE    ' → break
EXIT DO       ' → break
EXIT SUB      ' → return
EXIT FUNCTION ' → return <function_name>
```

3.7 Subroutines & Functions

SUB (no return value)

```
SUB Greet(name AS STRING)
    PRINT "Hello, " + name
END SUB
```

```
' Calling:
CALL Greet("World")
Greet "World"      ' CALL keyword is optional
Greet("World")     ' Parentheses also work
```

FUNCTION (returns a value)

```
FUNCTION Add(a AS INTEGER, b AS INTEGER) AS INTEGER
    Add = a + b      ' Return by assigning to function name
END FUNCTION
```

```
' Or with RETURN:
FUNCTION Add2(a AS INTEGER, b AS INTEGER) AS INTEGER
    RETURN a + b
END FUNCTION
```

```
DIM result AS INTEGER
result = Add(3, 4)
```

Function return mechanism: The codegen creates a local variable with the function's name, initialized to `None`. Assigning to the function name sets this variable. A `return <name>` is appended at the end.

Parameter passing: BYVAL and BYREF keywords are accepted by the parser. In the Rust codegen, all parameters are passed by value (clone semantics).

Global variable scoping: Global variables are accessed via `gv()` / `gs()` thread-local storage. All SUB/FUNCTION bodies automatically access globals through these functions without explicit declarations.

3.8 CREATE Blocks (GUI)

```
CREATE Form1 AS RForm
  Caption = "My App"
  Width = 640
  Height = 480

  CREATE Panel1 AS RPanel
    Align = 5

    CREATE Button1 AS RButton
      Caption = "OK"
      Left = 10
      Top = 10
      OnClick = HandleOK
    END CREATE
  END CREATE
END CREATE
```

Semantics:

- `CREATE X AS RType` generates `rp_comp_create("x", "RType")` plus `rp_comp_set` for the parent
- Property assignments inside `CREATE` become `rp_comp_set("x", "prop", value)`
- Method calls inside `CREATE` become `rp_comp_method("x", "method", &[args])`
- Nested `CREATE`s pass the parent automatically
- Builtin function calls inside `CREATE` blocks are NOT prefixed (they're detected via a known-builtins list)
- The `create_stack` tracks nesting for implicit parent assignment

Q-prefix backward compatibility: Types prefixed with `Q` (from RapidQ) are automatically normalized to `R` prefix in the codegen.

3.9 WITH Blocks

```
DIM form AS RForm
```

```
WITH form
```

```
    .Caption = "My Form"
```

```
    .Width = 800
```

```
    .Height = 600
```

```
END WITH
```

The parser maintains a `with_stack`. When a `.` is encountered at the start of an expression or statement, it's resolved to `with_stack[-1].member`.

3.10 IMPORT Statement

```
IMPORT "math" AS math
```

Generated as a comment in Rust. Data science functionality is provided via `RNum`, `RPlot`, and `RDataFrame` component types.

3.11 DECLARE Statement (DLL/FFI)

```
DECLARE SUB Sleep LIB "kernel32" ALIAS "Sleep" (ms AS LONG)
```

```
DECLARE FUNCTION GetTickCount LIB "kernel32" ALIAS "GetTickCount" () AS LONG
```

Translated to FFI calls via `libloading`. The shared library is loaded at runtime and functions are called through the `ffi.rs` module.

3.12 BIND Statement

```
BIND Button1.OnClick TO HandleClick
```

Generated as `button1.onclick = handleclick`.

3.13 PRINT Statement

```
PRINT "Hello"  
PRINT "A", "B", "C"  
PRINT "No newline";  
PRINT  
PRINT #1, "To file"
```

- ' With newline
- ' Comma-separated (space-separated in output)
- ' Semicolon suppresses newline
- ' Empty line
- ' File I/O (handled by builtins)

Generated as `rp_print(...)` calls in the runtime.

3.14 Operators

Operator	BASIC	Rust Equivalent
Addition	+	+
Subtraction	-	-
Multiplication	*	*
Division	/	/
Integer Division	\	/ (int)
Modulo	MOD	%
Exponentiation	^	.powf()
String Concatenation	& or +	format!()
Equal	=	==
Not Equal	<>	!=
Less Than	<	<
Greater Than	>	>
Less/Equal	<=	<=
Greater/Equal	>=	>=
Logical AND	AND	&&
Logical OR	OR	

Operator	BASIC	Rust Equivalent
Logical NOT	NOT	!
Logical XOR	XOR	^

Operator precedence (highest to lowest):

- 1. Unary (NOT , - , +)
- 2. Exponentiation (^)
- 3. Multiplication/Division (* , / , \ , MOD)
- 4. Addition/Subtraction (+ , - , &)
- 5. Comparison (< , > , <= , >=)
- 6. Equality (= , <>)
- 7. Logical AND
- 8. Logical OR, XOR

3.15 String Suffixes

BASIC-style identifier suffixes are accepted by the lexer:

- \$ — String (e.g., LEFT\$, MID\$, name\$)
- % — Integer
- # — Double
- & — Long
- ! — Single

These are **stripped during code generation** — they don't affect typing.

4. Preprocessor Directives

Handled in `compiler/preprocessor.py` before lexing.

Directive	Description	Behavior
<code>\$INCLUDE "file"</code>	Include external source file	Recursively preprocesses and inlines; detects circular includes

Directive	Description	Behavior
<code>\$DEFINE SYMBOL [value]</code>	Define text substitution	Whole-word replacement in non-string segments; default value = "1"
<code>\$UNDEF SYMBOL</code>	Remove a define	
<code>\$IFDEF SYMBOL</code>	Conditional: skip if not defined	Supports nesting
<code>\$IFNDEF SYMBOL</code>	Conditional: skip if defined	
<code>\$ELSE</code>	Toggle conditional branch	
<code>\$ENDIF</code>	End conditional block	
<code>\$MACRO NAME[(params)] = body</code>	Define parameterized macro	Expanded before <code>\$DEFINE</code> substitution
<code>\$APPTYPE GUI CONSOLE CGI</code>	Set application type	Passed through to codegen as <code>DirectiveNode</code>
<code>\$OPTIMIZE ON OFF</code>	Optimization hint	Pass-through (no effect)
<code>\$ESCAPECHARS ON OFF</code>	Escape character mode	Pass-through (no effect)
<code>\$TYPECHECK ON OFF</code>	Enable/disable strict type checking	Enables undeclared variable/function errors
<code>\$OPTION EXPLICIT</code>	Same as <code>\$TYPECHECK ON</code>	
<code>\$OPTION DIM <TYPE></code>	Default DIM type	Changes default from <code>DOUBLE</code>
<code>\$THEME <name></code>	Set FLTK theme (Rust only)	Applied at program start; available themes: <code>Classic</code> , <code>Aero</code> , <code>Metro</code> ,

Directive	Description	Behavior
		AquaClassic , Greybird , Blue , Dark , HighContrast , AUTO

Line preservation: The preprocessor replaces consumed directive lines with empty strings to preserve line numbers for error reporting.

\$DEFINE substitution rules:

- Only replaces whole words (uses `\b` word boundaries)
- Skips content inside quoted strings (splits by `"` , only substitutes in even-indexed segments)
- Sorted by length (longest first) to avoid partial replacements
- Inline comments in `$DEFINE` lines are stripped via `split("")`

5. Compiler Internals

5.1 Lexer (`compiler/lexer.py`)

307 lines. Tokenizes source into `Token(type, value, line, column)` tuples.

Token types (enum `TokenType`):

- **Keywords (40+):** DIM , AS , IF , THEN , ELSE , ELSEIF , END , FOR , TO , STEP , NEXT , WHILE , WEND , DO , LOOP , UNTIL , SELECT , CASE , SUB , FUNCTION , CALL , RETURN , EXIT , PRINT , INPUT , GOTO , GOSUB , IMPORT , CREATE , CONST , TYPE , DECLARE , LIB , ALIAS , WITH , EXTENDS , PROPERTY , SET , BYVAL , BYREF , BIND , CONSTRUCTOR , AND , OR , NOT , XOR , MOD , DEFSTR , DEFINT , etc.
- **Type keywords:** INTEGER , STRING , DOUBLE , SINGLE , BYTE , WORD , DWORD , LONG , INT64 , CURRENCY , POBJECT
- **Operators:** PLUS , MINUS , STAR , SLASH , BACKSLASH , CARET , AMPERSAND , EQ , NEQ , LT , LTE , GT , GTE
- **Symbols:** LPAREN , RPAREN , COMMA , COLON , SEMI , DOT
- **Literals:** NUMBER , STRING_LIT , IDENTIFIER
- **Special:** NEWLINE , EOF , DIRECTIVE

Tokenization method: Single compiled regex with named groups, applied via `finditer()` . Rules are ordered by priority (comments first, then directives, numbers, strings, identifiers, operators).

Number formats:

- Decimal: 123 , 3.14 , 1.5e10
- Hex: &H1F or &hFF → converted to 0x1f
- Octal: &O77 → converted to 0o77
- Binary: &B1010 → converted to 0b1010

Line continuation: `_` at end of line (before newline) is swallowed as whitespace.

CRITICAL LIMITATION: The lexer does NOT support `[` , `]` , `{` , or `}` characters. Workarounds: use component methods (RNum.arange, RDataFrame.loadfromcsv) or RUSTSTART/RUSTEND blocks for raw Rust code.

5.2 Parser (compiler/parser.py)

847 lines. Recursive-descent parser producing AST from token stream.

Key methods:

- `parse()` → ProgramNode (top-level)
- `parse_statement()` — dispatches to specific statement parsers based on current token
- `parse_expression()` → starts precedence climbing
- `parse_assignment_or_call()` — handles the ambiguous
IDENTIFIER [.member]* [= expr | args] pattern

Expression precedence (method call chain):

```
parse_expression → parse_logical_or → parse_logical_and → parse_equality  
→ parse_comparison → parse_term → parse_factor → parse_power  
→ parse_unary → parse_primary
```

Statement terminators: NEWLINE or COLON (`:` allows multiple statements per line).

Block parsing: `parse_block(end_tokens)` — consumes statements until one of the `end_tokens` is found at the current position. Used by IF, FOR, WHILE, DO, SELECT, SUB, FUNCTION, TYPE, WITH, CREATE.

The assignment-or-call ambiguity: When the parser sees an IDENTIFIER :

1. If followed by `(` → parse as ArrayAccessNode (statement level)
2. If followed by `.` → chain into MemberAccessNode

3. If followed by `=` → it's an `AssignmentNode`
4. Otherwise → it's a `CallStatementNode` with space-separated arguments

The array/function ambiguity in expressions: In `parse_primary()`, `IDENTIFIER(args)` is always parsed as `FunctionCallNode`. The codegen resolves whether it's actually array indexing.

5.3 AST (`rapidr-ast`)

Located in `crates/rapidr-ast/src/lib.rs`. Defines all AST node types as Rust enums and structs.

Expression nodes:

- `Identifier(name)` — variable or function name
- `Literal(value, type_name)` — string, number literal
- `BinaryOp(left, op, right)` — binary operation
- `UnaryOp(op, operand)` — unary operation
- `ArrayAccess(array, index)` — explicit array access
- `MemberAccess(obj, member)` — dot access
- `FunctionCall(name, args)` — function call OR array access (ambiguous)
- `MethodCall(obj, method, args)` — method call
- `RustBlock(code)` — raw Rust code (`RUSTSTART/RUSTEND`)

Statement nodes:

- `DimStatement`, `AssignmentStatement`, `IfStatement`, `ForStatement`, `WhileStatement`
- `DoLoopStatement`, `SelectCaseStatement`, `PrintStatement`
- `SubroutineDef`, `FunctionDef`, `CallStatement`, `MethodCallStatement`
- `ImportStatement`, `WithStatement`, `CreateStatement`
- `ReturnStatement`, `ExitStatement`, `DirectiveStatement`
- `ConstStatement`, `DeclareStatement`, `TypeStatement`, `BindStatement`

5.4 Code Generator (`rapidr-codegen-rust`)

Located in `crates/rapidr-codegen-rust/src/lib.rs` (~2,200 lines). Walks AST and emits Rust source code targeting `rapidr-runtime-core`.

Key state:

- `output` — String buffer for generated code
- `indent` — current Rust indentation depth
- `global_vars`, `arrays`, `consts` — collected from pre-passes

- `components` — set of CREATE'd component names
- `udts` — user-defined type names
- `sub_signatures` , `func_signatures` — SUB/FUNCTION parameter info
- `create_stack` — stack of current CREATE block names (for implicit property prefixing)

Three pre-passes before code generation:

1. Collect UDT type names
2. Collect global variables, arrays, constants, and CREATE components
3. Collect SUB/FUNCTION signatures, DECLARE, IMPORT modules

Generated Rust structure:

```
use rapidr_runtime_core::*;
// + thread_local globals, helper functions
fn main() {
    // Global initializations (gs/ga_init/rp_comp_create)
    // Top-level statements
    // gui_showmodal() if GUI app
}
// SUB/FUNCTION definitions as standalone fn's
```

5.5 Preprocessor (`rapidr-preprocessor`)

Located in `crates/rapidr-preprocessor/src/lib.rs` . Pure text-based preprocessing before lexing.

Key features:

- Recursive `$INCLUDE` with circular-include detection
- Nested `$IFDEF` / `$IFNDEF` with skip stack
- `$MACRO` with optional parameters
- `$THEME` for FLTK theming
- Line-number preservation

5.6 Diagnostics (`rapidr-diagnostics`)

Located in `crates/rapidr-diagnostics/src/lib.rs` . Defines `TextSpan` , `SourceLocation` , and `Diagnostic` types used throughout the compiler for error reporting with line/column information.

5.7 Symbol Table

The codegen maintains symbol information collected during pre-passes. Global variables, arrays, constants, component names, and SUB/FUNCTION signatures are all tracked for correct code emission.

5.8 Component Registry

The component registry is distributed across:

- `crates/rapidr-runtime-core/src/gui.rs` — widget creation (`gui_create_widget`)
- `crates/rapidr-runtime-core/src/object.rs` — property get/set/method dispatch
- `crates/rapidr-codegen-rust/src/lib.rs` — `is_component_type_name()` and `is_component_method_name()`

Covers 49+ component types: `RFORM` , `RBUTTON` , `RLABEL` , `REDIT` , `RCANVAS` , `RPANEL` , `RTIMER` , `RSTRINGGRID` , `RIMAGE` , `RCODEEDITOR` , `RMYSQL` , `RSQLITE` , `RSOCKET` , `RSERVERSOCKET` , `RHTTP` , `RNUM` , `RPLOT` , `RDATAFRAME` , `RDESIGNSURFACE` , `RTREEVIEW` , `RLISTVIEW` , `RSPLITTER` , `RTRACKBAR` , `RSCROLLBOX` , `RPOPUPMENU` , `RINI` , `RMEMORYSTREAM` , `RSTRINGLIST` , `RPRINTER` , `RCOLORDIALOG` , `RFONTDIALOG` , `RSTATUSBAR` , `RLINE` , `RICON` , `RIMAGELIST` , etc.

6. Runtime Library

The Rust runtime is in `crates/rapidr-runtime-core/src/` with modules:

Module	Lines	Description
<code>value.rs</code>	~300	Value enum (Int, Dbl, Str, Null) with arithmetic and comparison operators
<code>builtins.rs</code>	~450	100+ built-in BASIC functions (string, math, I/O, system)
<code>gui.rs</code>	~3,600	FLTK-based GUI — 49+ component types, event system, design surface
<code>object.rs</code>	~1,200	Component property get/set/method dispatch via <code>rp_comp_*</code> API
<code>database.rs</code>	~350	MySQL (<code>mysql</code> crate) and SQLite (<code>rusqlite</code> crate) components

Module	Lines	Description
network.rs	~400	TCP socket, server socket, HTTP client components
datascience.rs	~700	RNum (ndarray), RPlot (plotters), RDataFrame (polars)
file_io.rs	~200	RFileStream, RIni, RMemoryStream, RStringList implementations
ffi.rs	~200	DECLARE/DLL foreign function interface via libloading

6.1 Builtins (`builtins.rs`)

Implements 100+ BASIC functions as Rust functions operating on the `Value` type:

- **String functions:** `rp_chr`, `rp_asc`, `rp_left`, `rp_right`, `rp_mid`, `rp_len`, `rp_instr`, `rp_ucose`, `rp_lcase`, `rp_trim`, `rp_replace`, etc.
- **Math functions:** `rp_abs`, `rp_sin`, `rp_cos`, `rp_sqr`, `rp_rnd`, `rp_round`, `rp_ceil`, `rp_floor`, etc.
- **I/O functions:** `rp_print`, `rp_dir`, `rp_fileexists`, `rp_direxists`, `rp_mkdir`, `rp_rmdir`, `rp_kill`, `rp_rename`
- **System functions:** `rp_shell`, `rp_sleep`, `rp_timer`, `rp_date`, `rp_time`, `rp_environ`, `rp_command`
- **GUI functions:** `rp_showmessage`, `rp_messagebox`, `rp_rgb`

6.2 GUI (`gui.rs`)

The largest runtime module (~3,600 lines). Implements 49+ FLTK-based GUI components.

Component lifecycle:

1. `rp_comp_create(name, type)` — registers component in `GUI_COMPONENTS` thread-local
2. `gui_create_widget(name, type)` — creates FLTK widget, stores in `GUI_WIDGETS`
3. `rp_comp_set(name, prop, value)` — sets properties (before or after widget creation)
4. `gui_showmodal(form_name)` — starts FLTK event loop

Key component types:

- `RFORM` — FLTK Window with menu bar, status bar, timer support
- `RBUTTON` — Push button with hover/press color feedback
- `RLABEL`, `REDIT`, `RRICHEDIT` — Text display/input
- `RCANVAS` — Drawing surface with `pset/line/circle/fillrect/textout` methods
- `RSTRINGGRID` — Editable grid with column/row management

- **RCODEEDITOR** — Syntax-highlighted text editor with line numbers
- **RDESIGNSURFACE** — Visual form designer (used by the IDE)
- **RFONTDIALOG** — Font picker with preview
- **RCOLORDIALOG** — System color picker

6.3 Database (`database.rs`)

- **RMYSQL** — MySQL client via `mysql` crate. Connect, query, fetch rows/fields, iterate databases/tables.
- **RSQLITE** — SQLite via `rusqlite` crate. Same interface as RMYSQL.

6.4 Network (`network.rs`)

- **RSOCKET** — TCP client with connect/write/read, event callbacks
- **RSERVERSOCKET** — Threaded TCP server with per-client management, broadcast
- **RHTTP** — HTTP GET/POST client via `ureq` crate

6.5 Data Science (`datascience.rs`)

- **RNUM** — Numeric arrays via `ndarray` crate. Methods: `zeros` , `ones` , `arange` , `linspace` , `reshape` , `sum` , `mean` , `std` , `min` , `max` , `dot` , `transpose` , `sort` , `savetofile` , `loadfromfile` .
- **RPLOT** — Chart generation via `plotters` crate. Methods: `plot` , `scatter` , `bar` , `hist` , `pie` , `legend` , `clear` , `savetofile` , `saveto_buffer` .
- **RDATAFRAME** — DataFrames via `polars` crate. Methods: `loadfromcsv` , `savetocsv` , `loadfromjson` , `savetojson` , `head` , `tail` , `describe` , `sort` , `filter` , `groupby` , `addcolumn` , `deletecolumn` , `cell` , `setcell` , `query` , `tostring` , `tolist` .

7. Code Generation Patterns

Rust Code Generation (RapidR)

The Rust codegen (`rapidr-codegen-rust`) generates a standalone Rust project with `Cargo.toml` and `src/main.rs` . Generated code targets `rapidr-runtime-core` .

Global Variable Mechanism

Module-level DIM variables are stored in thread-local HashMap storage, accessed via helper functions:

DIM Counter AS INTEGER	→ gs("counter", v_int(0));	' Initialize
Counter = Counter + 1	→ gs("counter", &gv("counter") + &v_int(1));	' Assign
PRINT Counter	→ rp_print(&[gv("counter")], true);	' Read

Accessor functions:

- gs(name, value) — Set a global scalar
- gv(name) — Get a global scalar (returns owned Value)
- ga_init(name, size, default) — Initialize a global array
- ga_get(name, index) — Get global array element
- ga_set(name, index, value) — Set global array element

UDT variables at module level remain as native Rust structs (not stored in the HashMap), preserving field access syntax.

How a DIM becomes Rust

DIM x AS INTEGER	→ gs("x", v_int(0));	' Global (top-level)
DIM x AS INTEGER	→ let mut x = v_int(0);	' Local (inside SUB/FUNCTION)
DIM s AS STRING	→ gs("s", v_str(""));	' Global string
DIM f AS RForm	→ rp_comp_create("f", "RFORM");	' GUI component
DIM a(10) AS DOUBLE	→ ga_init("a", 11, v_dbl(0.0));	' Global array (0-10 = 11 el)
DIM p AS PersonType	→ let mut p = PersonType::default();	' UDT instance

How a CREATE block becomes Rust

CREATE Form1 AS RForm	→ rp_comp_create("form1", "RFORM");
Caption = "Test"	→ rp_comp_set("form1", "caption", v_str("Test"));
CREATE Btn AS RButton	→ rp_comp_create("btn", "RBUTTON");
Parent = Form1	→ rp_comp_set("btn", "parent", v_str("form1"));
Caption = "OK"	→ rp_comp_set("btn", "caption", v_str("OK"));
END CREATE	
END CREATE	

How a SUB becomes Rust

SUB Foo(x AS STRING)	→ fn foo(x: Value) {
DIM local AS STRING	→ let mut local = v_str("");
local = "hello"	→ local = v_str("hello");
PRINT local	→ rp_print(&[local.clone()], true);
END SUB	→ }

Global variables are accessed via `gv()` / `gs()` — no `global` declarations needed (thread-local storage handles sharing).

Changelog (April 2026)

Parser Fixes

- **Dot-member access after keywords:** `parse_postfix_expression` and `parse_primary` (WITH-dot case) now accept ANY token after `.`, not just `Identifier`. This fixes `Form1.Close`, `Form1.Show`, `ListBox1.Clear`, etc., where the member name is also a keyword.

Codegen Fixes

- **RTIMER registration:** `emit_create()` now emits `gui_register_timer("{name}")` after the `CREATE` block body for RTIMER components.

Runtime GUI Fixes (`gui.rs`)

- **Visibility default:** Components without an explicit `visible` property are now visible by default. Previously, missing/null `visible` was treated as 0 (hidden), making `RListBox`, `RCanvas`, `RImage`, etc. invisible. Now only explicitly set `"false"` or `"0"` hides a widget.
- **RButton visual feedback:** Replaced `FrameType`-based hover/press effects (invisible with themes) with color-based feedback. Added `Focus/Unfocus/KeyDown` handlers for keyboard interaction.
- **RCanvas inline color:** All drawing methods (`line`, `rect`, `fillrect`, `circle`, `ellipse`, `arc`, `drawtext`) accept an optional trailing color argument.
- **RCanvas drawtext:** Supports dual argument conventions — `drawtext(text, x, y)` and `drawtext(x, y, text)`.
- **RCanvas paint method:** Now calls `redraw_widget` instead of delegating to `fillrect`.

- **RCanvas onclick event:** Push handler now fires `rp_fire_event(name, "onclick")` .
- **Menu duplication fix:** `RMENUITEM` entries that have children (submenu headers like "&File") are no longer added to the `MenuBar` via `mb.add()` . Their children's full-path entries (e.g., "&File/&New") auto-create the submenu, preventing duplicate top-level menu entries.
- **RDESIGNSURFACE positioning:** Reads `left / top` from component properties instead of hardcoding (200, 200), enabling embedded MDI-style placement inside parent forms.
- **Component idempotency:** `rp_create_component` and `gui_create_widget` skip creation if the component/widget already exists, preventing duplicate widgets from DIM+CREATE patterns.

IDE Example (`ide.rr`)

- **MDI layout:** `DesignSurface` is now created inside the IDE form's `CREATE` block with `Left=4, Top=124, Width=832, Height=580` , embedding it directly in the center area rather than as a separate floating window.
- **Import Key :** Added `fltk::enums::Key` to imports for button keyboard handling.

CLI

- The `rapidr` binary can be installed globally via `cargo install --path crates/rapidr-cli` .

8. Known Limitations & Gotchas

Language Limitations

1. **No [,] , { , } in source code.** The lexer rejects these characters. Workaround: use component methods (`RNum.arange`, `RDataFrame.loadfromcsv`) or `RUSTSTART / RUSTEND` blocks for raw Rust.
2. **LEN() function** converts to string first — `LEN(x)` returns the length of the string representation.
3. **Array/function ambiguity.** `foo(i)` in expressions is always parsed as a function call. The codegen resolves it via symbol table lookup. If a variable isn't tracked (e.g., returned from a method, stored in `VARIANT`), it may be incorrectly emitted as a function call.
4. **All identifiers are lowercased.** `MyVar` and `MYVAR` are the same variable.
5. **No multi-line string literals.** Strings must be on a single line.
6. **No lambda/anonymous functions.** Use `CODEPTR(SubName)` for function references.
7. **GOTO/GOSUB recognized but not implemented** in codegen.

8. **Single = for both assignment and comparison.** Context determines meaning (statement = assignment, expression = comparison).

Compiler Gotchas (Rust Codegen)

1. `is_component_type_name()` and `is_component_method_name()` in `codegen-rust/src/lib.rs` must be updated when adding new component types or methods.
2. **Pre-pass ordering matters.** UDTs are collected first, then globals/arrays/consts, then SUB/FUNCTION signatures, then directives. Changing this order can break symbol resolution.
3. **String suffix stripping** — `LEFT$()` becomes `left`, `STR$()` becomes `str_func`.

Runtime Gotchas (Rust)

1. **FLTK event loop.** `gui_showmodal()` runs `app.run()` which blocks. The first form shown should be the main form.
2. **Thread-local storage.** All global variables and component state use `thread_local!`. Cross-thread GUI access is not supported.
3. **RStringGrid.AddRow** — takes variable positional string args. Column count must be set first.

9. How to Add New Features

Adding a New GUI Component (Rust)

1. `crates/rapidr-runtime-core/src/gui.rs` — Add widget creation in `gui_create_widget()` match arm
2. `crates/rapidr-runtime-core/src/object.rs` — Add default properties in `rp_comp_create()` match arm
3. `crates/rapidr-codegen-rust/src/lib.rs` — Add to `is_component_type_name()` and `is_component_method_name()`
4. **Add tests** in the appropriate crate

Adding a New Builtin Function (Rust)

1. `crates/rapidr-runtime-core/src/builtins.rs` — Implement the function
2. `crates/rapidr-codegen-rust/src/lib.rs` — Add to the builtin function emission logic
3. **Add tests**

Adding a New Keyword

- 1. `crates/rapidr-lexer/src/lib.rs` — Add to `TokenKind` enum and keyword matching
- 2. `crates/rapidr-parser/src/lib.rs` — Add parsing logic
- 3. `crates/rapidr-ast/src/lib.rs` — Create new AST node if needed
- 4. `crates/rapidr-codegen-rust/src/lib.rs` — Add code emission logic

Adding a New Preprocessor Directive

- 1. `crates/rapidr-preprocessor/src/lib.rs` — Add handling in the directive processing loop

10. Test Suite

Rust Tests

```
cargo test
```

Crate	Tests	Coverage
rapidr-codegen-rust	6	DIM, FOR, assignment, expression codegen
rapidr-lexer	7	Token types, keywords, operators, literals
rapidr-preprocessor	1	Directive handling
rapidr-parser	15	AST generation for all language constructs
rapidr-diagnostics	1	Span and diagnostic types
rapidr-ast	9	AST node validation
rapidr-cli	31	CLI commands, example compilation smoke tests

End of manual. Keep this file updated as the compiler evolves.